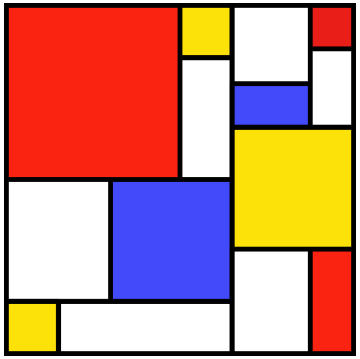
SLiM

Workshop Series



Eidos Overview

Benjamin C. Haller

Messer Lab, Cornell University

Data types

- `NULL`: no explicit value

Data types

- `NULL`: no explicit value
- `logical`: a Boolean true/false value (`T`/`F`)

Data types

- `NULL`: no explicit value
- `logical`: a Boolean true/false value (`T`/`F`)
- `integer`: a 64-bit signed integer (`10`, `-27`)

Data types

- `NULL`: no explicit value
- `logical`: a Boolean true/false value (`T`/`F`)
- `integer`: a 64-bit signed integer (`10`, `-27`)
- `float`: a floating-point number (`10.0`, `-2.7`)

Data types

- `NULL`: no explicit value
- `logical`: a Boolean true/false value (`T`/`F`)
- `integer`: a 64-bit signed integer (`10`, `-27`)
- `float`: a floating-point number (`10.0`, `-2.7`)
- `string`: a sequence of characters (`"foo"`)

Data types

- `NULL`: no explicit value
- `logical`: a Boolean true/false value (`T`/`F`)
- `integer`: a 64-bit signed integer (`10`, `-27`)
- `float`: a floating-point number (`10.0`, `-2.7`)
- `string`: a sequence of characters (`"foo"`)
- `object`: an instance of a class (`Individual`)

Operators

- Arithmetic: `+`, `-`, `*`, `/`, `%`, `^`, `:`

```
6 + 2*7
```

Operators

- Arithmetic: `+`, `-`, `*`, `/`, `%`, `^`, `:` `6 + 2*7`
- Logical: `&`, `|`, `!` `T & !F`

Operators

- Arithmetic: `+`, `-`, `*`, `/`, `%`, `^`, `:` `6 + 2*7`
- Logical: `&`, `|`, `!` `T & !F`
- Comparison: `<`, `>`, `<=`, `>=`, `==`, `!=` `2+2 == 4`

Operators

- Arithmetic: `+`, `-`, `*`, `/`, `%`, `^`, `:` — `6 + 2*7`
- Logical: `&`, `|`, `!` — `T & !F`
- Comparison: `<`, `>`, `<=`, `>=`, `==`, `!=` — `2+2 == 4`
- Assignment: `=` — `x = 8`

Operators

- Arithmetic: `+`, `-`, `*`, `/`, `%`, `^`, `:` — `6 + 2*7`
- Logical: `&`, `|`, `!` — `T & !F`
- Comparison: `<`, `>`, `<=`, `>=`, `==`, `!=` — `2+2 == 4`
- Assignment: `=` — `x = 8`
- Precedence and function call: `()` — `(6+2) * 7`

Operators

- Arithmetic: `+`, `-`, `*`, `/`, `%`, `^`, `:` — `6 + 2*7`
- Logical: `&`, `|`, `!` — `T & !F`
- Comparison: `<`, `>`, `<=`, `>=`, `==`, `!=` — `2+2 == 4`
- Assignment: `=` — `x = 8`
- Precedence and function call: `()` — `(6+2) * 7`
- Subset: `[]` — `x[5]`

Operators

- Arithmetic: `+`, `-`, `*`, `/`, `%`, `^`, `:` — `6 + 2*7`
- Logical: `&`, `|`, `!` — `T & !F`
- Comparison: `<`, `>`, `<=`, `>=`, `==`, `!=` — `2+2 == 4`
- Assignment: `=` — `x = 8`
- Precedence and function call: `()` — `(6+2) * 7`
- Subset: `[]` — `x[5]`
- Property access and method call: `.` — `foo.bar`

Operators

- Arithmetic: `+`, `-`, `*`, `/`, `%`, `^`, `:` — `6 + 2*7`
- Logical: `&`, `|`, `!` — `T & !F`
- Comparison: `<`, `>`, `<=`, `>=`, `==`, `!=` — `2+2 == 4`
- Assignment: `=` — `x = 8`
- Precedence and function call: `()` — `(6+2) * 7`
- Subset: `[]` — `x[5]`
- Property access and method call: `.` — `foo.bar`
- Ternary conditional: `?else`

Objects, properties, methods

- Objects represent **entities**:
  - e.g., individuals, mutations, subpopulations

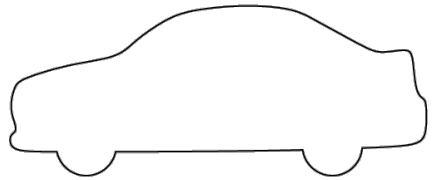
Objects, properties, methods

- Objects represent **entities**:
  - e.g., individuals, mutations, subpopulations
- Objects have **properties**:
  - attributes like `age`, `sex`, `spatialPosition`
  - `individual.age` returns the age of `individual`
  - `individual.age = 10;` changes its age

Objects, properties, methods

- Objects represent **entities**:
  - e.g., individuals, mutations, subpopulations
- Objects have **properties**:
  - attributes like `age`, `sex`, `spatialPosition`
  - `individual.age` returns the age of `individual`
  - `individual.age = 10;` changes its age
- Objects have **methods**:
  - methods perform complex operations
  - `individual.containsMutations(muts)`

Objects, properties, methods



object class: `Car`

object: `my_car`

properties:

```
my_car.make == "Honda"
my_car.model == "CR-V"
my_car.color == "red"
```

method calls:

```
my_car.driveTo(the_mall)
```

object: `i_wish`

properties:

```
i_wish.make == "Ferrari"
i_wish.model == "F12"
i_wish.color == "yellow"
```

method calls:

```
i_wish.driveTo(California)
```

Everything is a vector

- All values in Eidos are vectors
- A single integer, `10`, is really a vector, `10`
- A vector of exactly one value is a **singleton**

Everything is a vector

- All values in Eidos are vectors
- A single integer, `10`, is really a vector, `10`
- A vector of exactly one value is a **singleton**
- Most operations in Eidos are **vectorized**:
  - operators like `+`, `-`, `*`, `/`
  - property access on objects, like `foo.bar`
  - method calls on objects, like `foo.bar()`
- This allows Eidos to be fast!

Everything is a vector

- How do vectorized operators work?
  - 1×1 operations: `1 + 1` → `2`
  - 1×N operations: `1 + 1:3` → `2 3 4`
  - N×1 operations: `1:3 - 1` → `0 1 2`
  - N×N operations: `1:3 * 2:4` → `2 6 12`
  - operands must be *conformable*:
    - `1:3 * 1:5` → error

Everything is a vector

- What about properties?
  - suppose `inds` is a vector of five individuals
  - 1×1 assignment: `inds[0].age = 2`
  - N×1 assignment: `inds.age = 2`
  - N×N operations: `inds.age = 1:5`
  - assignments must, again, be *conformable*:
    - `inds.age = 1:3` → error

Everything is a vector

- What about methods?
  - suppose `inds` is a vector of five individuals
  - – `inds.method(...)` is called **five** times
    - each object does the same work

Everything is a vector

- What about methods?
  - suppose `inds` is a vector of five individuals
  - `– inds.method(...)` is called **five** times
    - each object does the same work
  - `+ inds.method(...)` is called **one** time
    - this is called a "class method" in Eidos
    - like a "static member function" in C++
    - processes all the targets in one call
    - example: `haplosomes.outputVCF(...)`

Control flow

- **if–else**: conditional execution
  - `if (condition) statement; else statement;`

Control flow

- **if–else**: conditional execution
  - `if (condition) statement; else statement;`
- **while**: loop on a condition, 0+ times
  - `while (condition) statement;`

Control flow

- **if–else**: conditional execution
  - `if (condition) statement; else statement;`
- **while**: loop on a condition, 0+ times
  - `while (condition) statement;`
- **do–while**: loop on a condition, 1+ times
  - `do statement; while (condition);`

Control flow

- **if–else**: conditional execution
  - `if (condition) statement; else statement;`
- **while**: loop on a condition, 0+ times
  - `while (condition) statement;`
- **do–while**: loop on a condition, 1+ times
  - `do statement; while (condition);`
- **for**: loop over the values in a vector
  - `for (i in vector) statement;`

Joint `for` loops

- You can loop over more than one vector at the same time, in synchrony!
- This is called a "joint `for` loop"
  -
```
for (a in 1:3, b in c("foo", "bar", "baz"))
{
   catn("a = " + a + ", b = " + b);
}

a = 1, b = foo
a = 2, b = bar
a = 3, b = baz
```
- The vectors must be the same size

Compound statements

- Compound statements use braces, `{ ... }`

```
if (condition)
{
   a;
   b;
   c;
}
else
{
   d;
   e;
}
```

```
while (condition)
{
   a;
   b;
}

for (i in 1:100)
{
   a;
   b;
}
```

Compound statements

- Compound statements use braces, `{ ... }`

```
if (condition)
{
   a;
   b;
   c;
}
else
{
   d;
   e;
}
```

```
while (condition)
{
   a;
   b;
}

for (i in 1:100)
{
   a;
   b;
}
```

- Indentation is not significant (unlike Python)

Built-in functions

Built-in functions

- **Math**: `abs()`, `ceil()`, `log()`, `setUnion()`, ...

Built-in functions

- **Math**: `abs()`, `ceil()`, `log()`, `setUnion()`, ...
- **Statistics**: `max()`, `mean()`, `sd()`, `cov()`, ...

Built-in functions

- **Math**: `abs()`, `ceil()`, `log()`, `setUnion()`, ...
- **Statistics**: `max()`, `mean()`, `sd()`, `cov()`, ...
- **Distributions**: `rnorm()`, `rpois()`, `runif()`, ...

Built-in functions

- **Math**: `abs()`, `ceil()`, `log()`, `setUnion()`, ...
- **Statistics**: `max()`, `mean()`, `sd()`, `cov()`, ...
- **Distributions**: `rnorm()`, `rpois()`, `runif()`, ...
- **Vectors**: `c()`, `rep()`, `seq()`, `sample()`, ...

Built-in functions

- **Math**: `abs()`, `ceil()`, `log()`, `setUnion()`, ...
- **Statistics**: `max()`, `mean()`, `sd()`, `cov()`, ...
- **Distributions**: `rnorm()`, `rpois()`, `runif()`, ...
- **Vectors**: `c()`, `rep()`, `seq()`, `sample()`, ...
- **Values**: `all()`, `any()`, `identical()`, `sort()`, ...

Built-in functions

- **Math**: `abs()`, `ceil()`, `log()`, `setUnion()`, ...
- **Statistics**: `max()`, `mean()`, `sd()`, `cov()`, ...
- **Distributions**: `rnorm()`, `rpois()`, `runif()`, ...
- **Vectors**: `c()`, `rep()`, `seq()`, `sample()`, ...
- **Values**: `all()`, `any()`, `identical()`, `sort()`, ...
- **Output**: `cat()`, `print()`, `paste()`, `str()`, ...

Built-in functions

- **Math**: `abs()`, `ceil()`, `log()`, `setUnion()`, ...
- **Statistics**: `max()`, `mean()`, `sd()`, `cov()`, ...
- **Distributions**: `rnorm()`, `rpois()`, `runif()`, ...
- **Vectors**: `c()`, `rep()`, `seq()`, `sample()`, ...
- **Values**: `all()`, `any()`, `identical()`, `sort()`, ...
- **Output**: `cat()`, `print()`, `paste()`, `str()`, ...
- **Types**: `isFloat()`, `asFloat()`, ...

Built-in functions

- **Math**: `abs()`, `ceil()`, `log()`, `setUnion()`, ...
- **Statistics**: `max()`, `mean()`, `sd()`, `cov()`, ...
- **Distributions**: `rnorm()`, `rpois()`, `runif()`, ...
- **Vectors**: `c()`, `rep()`, `seq()`, `sample()`, ...
- **Values**: `all()`, `any()`, `identical()`, `sort()`, ...
- **Output**: `cat()`, `print()`, `paste()`, `str()`, ...
- **Types**: `isFloat()`, `asFloat()`, ...
- **Filesystem**: `readFile()`, `writeFile()`, ...

Essential built-in functions

- `c()`: Create a vector by concatenation
  - `c(2:5, 7, 9:11)` produces `2 3 4 5 7 9 10 11`

Essential built-in functions

- `c()`: Create a vector by concatenation
  - `c(2:5, 7, 9:11)` produces `2 3 4 5 7 9 10 11`
- `print()`, `cat()`, `catn()`: produce output
  - for printing one value / variable, `print()` is better
  - for formatted output, `cat()` / `catn()` is better

Essential built-in functions

- `c()`: Create a vector by concatenation
  - `c(2:5, 7, 9:11)` produces `2 3 4 5 7 9 10 11`
- `print()`, `cat()`, `catn()`: produce output
  - for printing one value / variable, `print()` is better
  - for formatted output, `cat()` / `catn()` is better
- `defineConstant()`: define a new constant
  - `defineConstant("X", 1:10)` defines `X` to be `1:10`
  - `X` will be defined globally and permanently

Eidos reference sheet

Eidos Reference Sheet

17 April 2025

Types (in promotion order):

- `NULL`: no explicit value
- `logical`: true/false values
- `integer`: whole numbers
- `float`: real numbers
- `string`: characters
- `object`: Eidos objects, such as SLiM objects

Constants:

- `E`: e (2.7182...) (float)
- `PI`: π (3.1415...) (float)
- `F`: false (logical)
- `T`: true (logical)
- `INF`: infinity (float)
- `NAN`: not a number (float)
- `NULL`: a `NULL`-type value

Operators (precedence order):

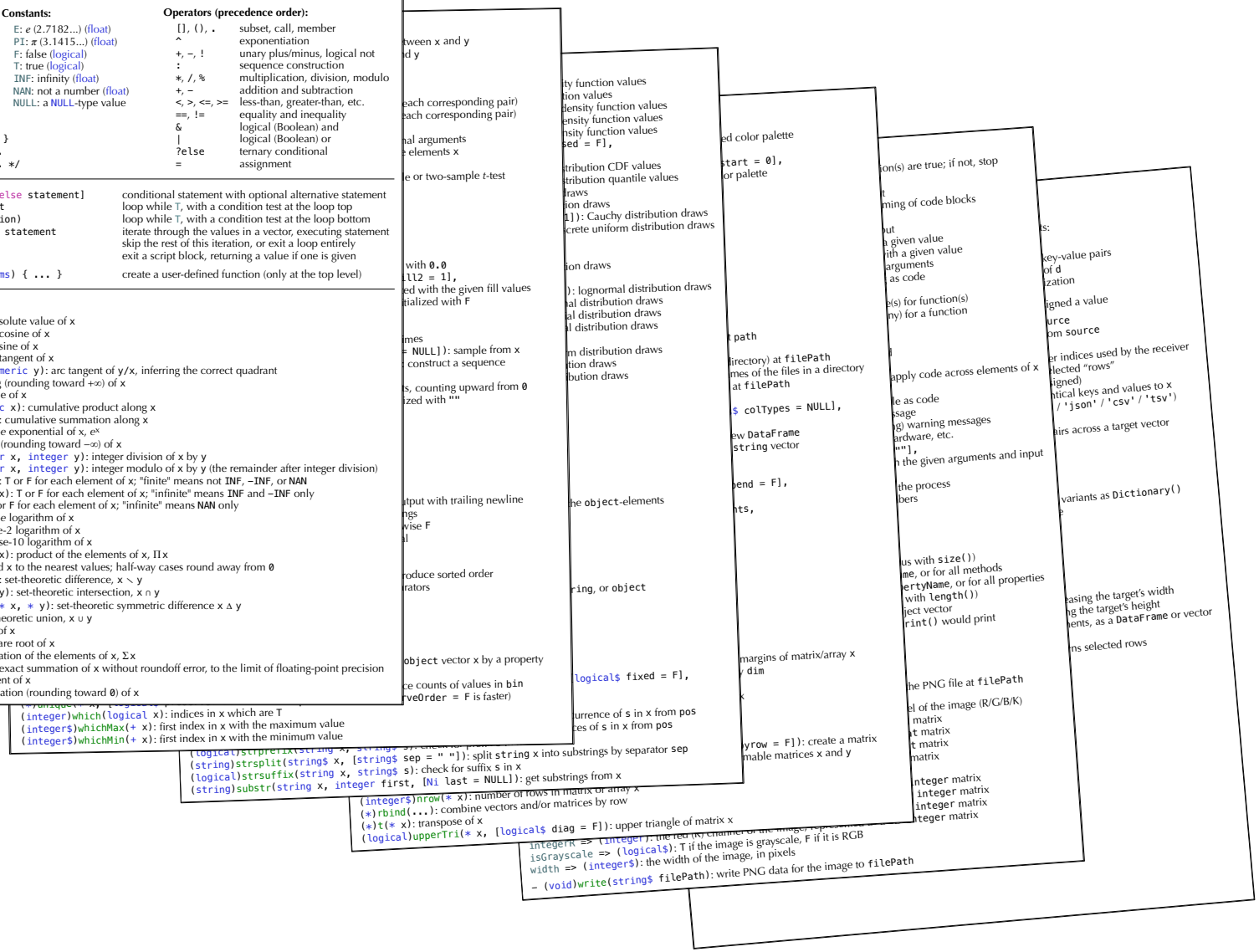
`[], (), .`	subset, call, member
`^`	exponentiation
`+, -, !`	unary plus/minus, logical not
`:`	sequence construction
`*, /, %`	multiplication, division, modulo
`+, -`	addition and subtraction
`<, >, <=, >=`	less-than, greater-than, etc.
`==, !=`	equality and inequality
`&`	logical (Boolean) and
`|`	logical (Boolean) or
`?else`	ternary conditional
`=`	assignment

Empty statement:	`;`
Compound statement:	`{ ... }`
Single-line comment:	`// ...`
Block comment:	`/* ... */`

`if (condition) statement [else statement]`	conditional statement with optional alternative statement
`while (condition) statement`	loop while `T`, with a condition test at the loop top
`do statement while (condition)`	loop while `T`, with a condition test at the loop bottom
`for (identifier in vector) statement`	iterate through the values in a vector, executing statement
`next / break`	skip the rest of this iteration, or exit a loop entirely
`return [return-value]`	exit a script block, returning a value if one is given
`function (return)name(params) { ... }`	create a user-defined function (only at the top level)

Math:

- `(numeric)abs(numeric x)`: absolute value of `x`
- `(float)acos(numeric x)`: arc cosine of `x`
- `(float)asin(numeric x)`: arc sine of `x`
- `(float)atan(numeric x)`: arc tangent of `x`
- `(float)atan2(numeric x, numeric y)`: arc tangent of `y/x`, inferring the correct quadrant
- `(float)ceil(float x)`: ceiling (rounding toward +∞) of `x`
- `(float)cos(numeric x)`: cosine of `x`
- `(numeric)cumProduct(numeric x)`: cumulative product along `x`
- `(numeric)cumSum(numeric x)`: cumulative summation along `x`
- `(float)exp(numeric x)`: base-e exponential of `x`, e^x
- `(float)floor(float x)`: floor (rounding toward −∞) of `x`
- `(integer)integerDiv(integer x, integer y)`: integer division of `x` by `y`
- `(integer)integerMod(integer x, integer y)`: integer modulo of `x` by `y` (the remainder after integer division)
- `(logical)isFinite(float x)`: `T` or `F` for each element of `x`; "finite" means not `INF`, `-INF`, or `NAN`
- `(logical)isInfinite(float x)`: `T` or `F` for each element of `x`; "infinite" means `INF` and `-INF` only
- `(logical)isNAN(float x)`: `T` or `F` for each element of `x`; "infinite" means `NAN` only
- `(float)log(numeric x)`: base-e logarithm of `x`
- `(float)log2(numeric x)`: base-2 logarithm of `x`
- `(float)log10(numeric x)`: base-10 logarithm of `x`
- `(numeric$)product(numeric x)`: product of the elements of `x`, Π `x`
- `(float)round(float x)`: round `x` to the nearest values; half-way cases round away from `0`
- `(*)setDifference(* x, * y)`: set-theoretic difference, `x` ∖ `y`
- `(*)setIntersection(* x, * y)`: set-theoretic intersection, `x` ∩ `y`
- `(*)setSymmetricDifference(* x, * y)`: set-theoretic symmetric difference `x` Δ `y`
- `(*)setUnion(* x, * y)`: set-theoretic union, `x` ∪ `y`
- `(float)sin(numeric x)`: sine of `x`
- `(float)sqrt(numeric x)`: square root of `x`
- `(numeric$)sum(lif x)`: summation of the elements of `x`, Σ `x`
- `(float$)sumExact(float x)`: exact summation of `x` without roundoff error, to the limit of floating-point precision
- `(float)tan(numeric x)`: tangent of `x`
- `(float)trunc(float x)`: truncation (rounding toward `0`) of `x`



Function signatures

- Function signatures describe calling conventions
  - parameter types and names, return type
- For example, for `runif()`:

```
(float)runif(integer$ n, [numeric min = 0], [numeric max = 1])
```

Function signatures

- Function signatures describe calling conventions
  - parameter types and names, return type
- For example, for `runif()`:

```
(float)runif(integer$ n, [numeric min = 0], [numeric max = 1])
```

  - Why do we need this cryptic syntax?
    - Understanding the **reference sheets**
    - Creating **user-defined functions**

Function signatures

```
(float)runif(integer$ n, [numeric min = 0], [numeric max = 1])
```

- The basic outline is:
  - `(return-type)functionName(parameters)`
- A `$` indicates a required *singleton*
- Brackets, `[ ]`, indicate *optional* parameters
- An `=` indicates a *default value*
- Type `numeric` means `integer` or `float`
- Type `void` means no return value

Function signatures

- A type of `*` means "any type"

  `(integer$)size(* x)`

- A type of `+` means "any non-object type"

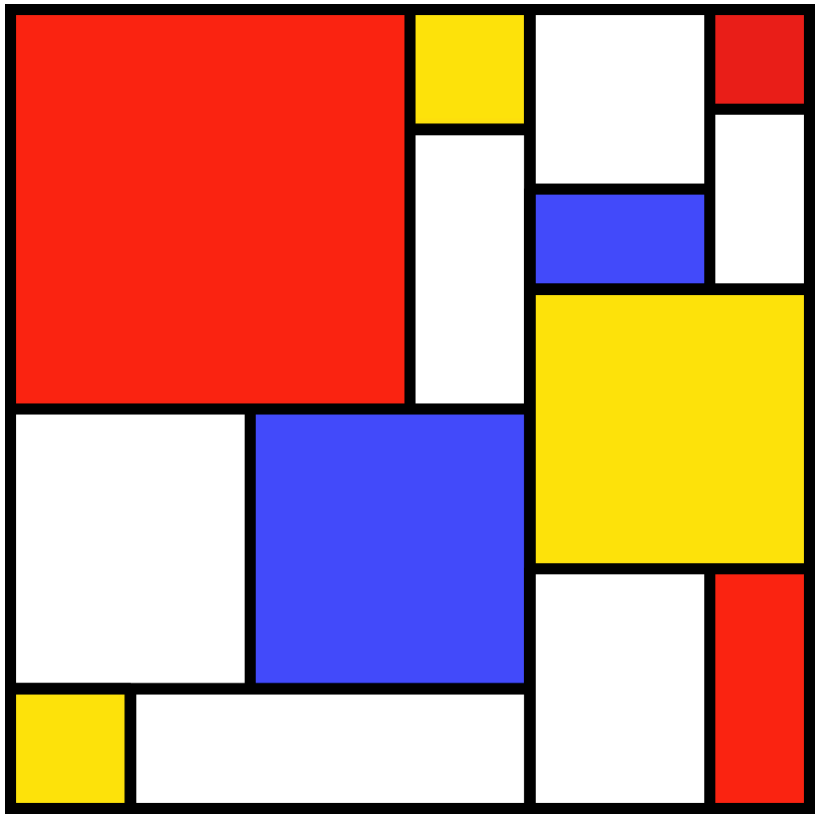
  `(integer)asInteger(+ x)`

- Complex types are represented with single letters

  `(numeric$)sum(lif x)`

- An ellipsis, `...`, indicates *variable* parameters

  `(*)c(...)`



Copyright © 2020–2025 Benjamin C. Haller. All Rights Reserved.